

Numpy Library

Numeric Python $\Rightarrow$ NumPy

Fast computation with n -dimⁿ array

↓

2

2

n-d array

```
import numpy as np
```

np. $\underbrace{\hspace{2cm}} (\hspace{1cm})$

Numpy array vs list

Lists are a lot slower than np array

Array (np)

List

Fixed size

expandable

(new created)

homogenous array

heterogenous

math $\hat{f}_n$ supported + provided

25 Such Support

Creating a numpy array

$a = \text{np.array}(\text{---}) \Rightarrow \text{nd array}$

$\text{type}(a) \Rightarrow \text{nd array}$

1-D array $\Rightarrow$ vector

2-D array $\Rightarrow$ matrix

3-D array $\Rightarrow$ tensor

Creation of NumPy Arrays

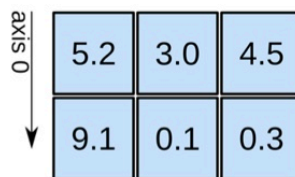
- $\text{Np.array}(\text{Sequence})$

1D array



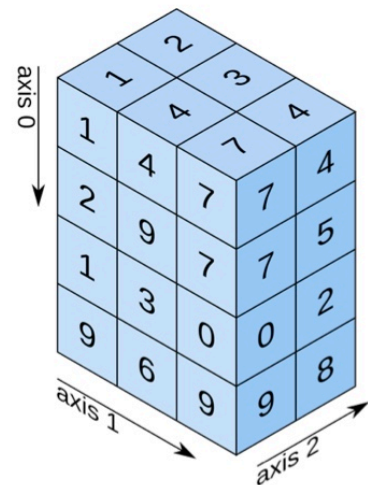
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

numpy type name	array interface	numpy type code	numpy type codes	array interface	numpy type names
-----------------	-----------------	-----------------	------------------	-----------------	------------------

`dtype('float64')` == `dtype('<f8')` == `dtype('d')`

(Arrows from `<`, `f`, and `8` point to the endianness, type, and size in bytes columns respectively)

'=' native	'?' boolean	'?' 'b1' 'bool'
'<' little endian	'b' (signed) byte	'b' 'i1' 'int8'
'>' big endian	'B' unsigned byte	'B' 'u1' 'uint8'
' ' not applicable	'i' (signed) integer	'h' '<i2' 'int16'
	'u' unsigned integer	'H' '<u2' 'uint16'
	'f' floating-point	'i' '<i4' 'int32'
	'c' complex-floating point	'I' '<u4' 'uint32'
	'm' timedelta	'l' '<i4' 'int32'
	'M' datetime	'L' '<u4' 'uint32'
	'O' (Python) objects	'q' '<i8' 'int64'
	'S', 'a' zero-terminated bytes	'Q' '<u8' 'uint64'
	'U' Unicode string	'e' '<f2' 'float16'
	'V' raw data (void)	'f' '<f4' 'float32'
		'd' '<f8' 'float64'
		'F' '<c8' 'complex64'
		'D' '<c16' 'complex128'
		'm' '<m8[D]' 'timedelta64[D]
		'M' '<M8[D]' 'datetime64[D]
		'c' ' S1' 1 byte (e.g. b'a')
		'S' '<S4' 4 bytes
		'U' '<U4' 4-char str
		'O' ' O' 'object'
		'V' ' V0' np.void

Creating numpy arrays

1] np.array

with type

without type
astype

2] np.arange

3] np.ones ✓

4] np.zeros ✓

5] np.random.random

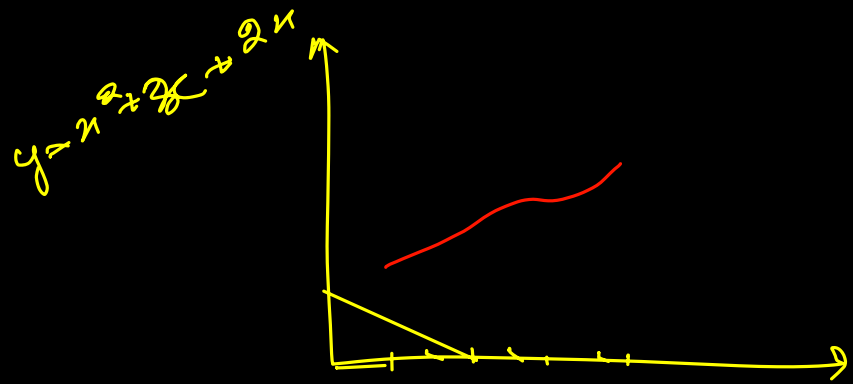
6] np.linspace

np.identity



0, 5, 10

0



$$y = x^2 + 2x + 2x$$

np.identity(5)

A 5x5 grid of boxes representing an identity matrix. Each box contains a number. The diagonal elements (top-left to bottom-right) are 1, and all other elements are 0. The boxes are arranged in 5 rows and 5 columns.

1.	0.	0.	0.	0.
0.	1.	0.	0.	0.
0.	0.	1.	0.	0.
0.	0.	0.	1.	0.
0.	0.	0.	0.	1.

Array attributes

ndim

shape

size

itemsize

reshape

1, 2, 3, 4, 5, 6, 7, 8, 9

3x3

2x5 $\Rightarrow$ 10

To change shape of already created
np array we use `astype`

Scalar Operation

$a1 + 2$

$-$

$\div$

$*$

$//$

$\%$

$\&$

$\Rightarrow$ every element will
gets increased by
2

a_1	a_2	a_3
a_4	a_5	a_6
a_7	a_8	a_9

 $*$

b_1	b_2	b_3
b_4	b_5	b_6
b_7	b_8	b_9

 $=$

a_1b_1	a_2b_2	a_3b_3
a_4b_4	a_5b_5	a_6b_6
a_7b_7	a_8b_8	a_9b_9

Dot Product

$$\begin{aligned}
 & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} (1 \cdot 5) + (2 \cdot 7) & (1 \cdot 6) + (2 \cdot 8) \\ (3 \cdot 5) + (4 \cdot 7) & (3 \cdot 6) + (4 \cdot 8) \end{bmatrix} \\
 & = \begin{bmatrix} 5+14 & 6+16 \\ 15+28 & 18+32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}
 \end{aligned}$$

$$A_{r_1 \times c_1}$$

$$B_{r_2 \times c_2}$$

$$c_1 = r_2$$

$$= R_{r_1 \times c_2}$$

Slicing & indexing

Broadcasting

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \quad (3,3) \quad * \quad \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array} \quad (3,) \text{ or } (1,3) \quad = \quad \begin{array}{|c|c|c|} \hline -1 & 0 & 3 \\ \hline -4 & 0 & 6 \\ \hline -7 & 0 & 9 \\ \hline \end{array} \quad (3,3)$$

multiplying several
columns at once

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \quad (3,3) \quad / \quad \begin{array}{|c|c|c|} \hline 3 & 3 & 3 \\ \hline 6 & 6 & 6 \\ \hline 9 & 9 & 9 \\ \hline \end{array} \quad (3,1) \quad = \quad \begin{array}{|c|c|c|} \hline .3 & .7 & 1. \\ \hline .6 & .8 & 1. \\ \hline .8 & .9 & 1. \\ \hline \end{array} \quad (3,3)$$

row-wise
normalization

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline \end{array} \quad (3,) \text{ or } (1,3) \quad * \quad \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 2 & 2 & 2 \\ \hline 3 & 3 & 3 \\ \hline \end{array} \quad (3,1) \quad = \quad \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 2 & 4 & 6 \\ \hline 3 & 6 & 9 \\ \hline \end{array} \quad (3,3)$$

outer product